AI-Powered News Translation from Kannada to English

A complete guide from idea to working application: options, architecture, AI pipeline, research design, and ready-to-paste Antigravity prompts

Prepared 10 November 2026. I checked model cards, papers, datasets and pricing pages on this date. Anything I could not confirm is marked ⚠. Verify those items yourself before relying on or citing them. For papers I often saw only the abstract or key sections, so read each paper before you cite it.

0. The short version

- **One application, one pipeline.** Kannada article → clean text → Kannada-to-English translation → English headline, summary and key points → automatic verification against both the translation and the Kannada original → English presentation shown next to the original.
- **Translate first, then summarize.** A 2024 study covering 39 source languages found that a simple translate-then-summarize pipeline consistently beat end-to-end systems (Varab & Hardmeier, arXiv 2409.00414). English summarizers and fact-checkers are also much stronger than Kannada ones.
- **Translation engine:** IndicTrans2 (open, free, self-hosted) as the default. Sarvam-Translate and an LLM API become comparison systems you can swap in.
- **Summaries:** an LLM with a strict JSON output where every claim must cite sentences from the translation, followed by automated checks that catch invented names, numbers and claims.
- **News sources:** RSS feeds from outlets whose terms allow it, plus user-pasted Kannada text. Show original and translation side by side.
- **Stack:** Next.js + FastAPI + PostgreSQL, with a background worker for the AI steps. It can run at zero cost using free GPU notebooks for batch work.
- **Research value:** modest but real, if you are honest about it. Your contribution is a Kannada news-domain evaluation of translate-then-summarize with a faithfulness-verification layer. It is not a new translation model.

1. The possible implementations, compared

There are six real design decisions. For each, here are the alternatives and my recommendation.

1.1 Decision 1: which translation engine?

Option	What it is	Strengths	Weaknesses
A. IndicTrans2 (AI4Bharat)	Open sentence-level NMT for 22 Indian languages; 1B base and 200M distilled models	Strong published Indic results; MIT-licensed checkpoints; small enough to run locally (about 4.5 GB VRAM for 1B and about 950 MB for the 200M model, per a community server guide); mature toolkit	Trained on single sentences with a 256-token limit, so you must split and re-join text; no document context; needs trust_remote_code in Hugging Face
B. NLLB-200 (Meta)	One model covering 200 languages	Kannada included; several sizes (including distilled ones)	Weights are typically non-commercial (CC-BY-NC ⚠ confirm on the model card); the IndicTrans2 paper reports IndicTrans2 beating it on Indic languages, so treat it as a baseline
C. Sarvam-Translate	Gemma3-4B-IT fine-tuned for 22 Indian languages; document-level; 8k-token context; open weights plus a paid API	Handles long, structured text without sentence splitting	4B parameters means a GPU for local use; licence ⚠ (one reviewer reports GPL, and Gemma terms may also apply); one independent reviewer found a clear mistranslation (a title) in the vendor's own showcase example

Option	What it is	Strengths	Weaknesses
D. General LLM API (for example Gemini)	Prompt a large LLM to translate	Good context handling and fluency; no infrastructure	Rate limits and cost; free-tier content may be used to improve the provider's products ⚠; non-deterministic; may paraphrase or smooth over text, which hurts fidelity
E. Hybrid	IndicTrans2 as primary, a second system only for checking or comparison	Best accuracy per rupee; doubles as your research experiment	More code

Recommendation: build A as the production default behind a pluggable interface, so that C and D can be dropped in and compared. That interface is also your research experiment.

1.2 Decision 2: should summarization happen before or after translation?

Pipeline	How it works	Verdict
P1: Translate → Summarize	Translate the whole Kannada article to English, then summarize the English	**Recommended.** Uses the strongest tools for each step, and the English summary can be fact-checked against English text
P2: Summarize → Translate	Summarize in Kannada, then translate the short summary	Fewer tokens to translate, but Kannada summarization tools and Kannada fact-checkers are weak. Keep as a baseline
P3: One-shot LLM	Give the LLM Kannada, ask for an English summary	Simplest code, but a black box: you cannot tell whether an error came from translation or summarization. Keep as a baseline

What the evidence says: Varab & Hardmeier (2024) found translate-then-summarize beat end-to-end designs across 39 source languages into English. The PMIndiaSum paper (EMNLP 2023) benchmarked translate-and-summarize, summarize-and-translate and fine-tuning for Indian languages, and reported that the best order depends on language resource levels. A 2025 thesis on English-to-Hindi legal summaries found summarize-then-translate worked better, but that is the opposite direction (the target is the low-resource language). My inference, not a proven result for Kannada: when the target is English, doing the Kannada-specific work first and summarizing in English is the safer order.

The real weakness of P1 is error propagation: a mistranslated name or number flows into the summary, and an English-only fact-checker cannot see it. Section 6 fixes this by also verifying key facts against the original Kannada.

1.3 Decision 3: use an LLM for summaries?

Yes, but with guardrails. The options:

- **Extractive** (select the most important sentences using embeddings or TextRank): zero invented facts, but choppy and cannot write a headline.
- **Fine-tuned seq2seq** (BART-style models): cheap and local, but short input limits and older quality.
- **LLM, abstractive, with a strict schema:** best headline, summary and key points in one call. The risk is hallucination, which you control with evidence citations, automatic verification and an extractive fallback.

Recommendation: LLM abstractive generation, verified, with extractive fallback when verification fails.

1.4 Decision 4: where do articles come from?

- **RSS feeds:** the cleanest, most defensible route. Gives titles, links, dates, often section names.
- **News APIs:** useful for discovery, but they usually give only headlines or snippets and have licence and rate limits ⚠ check each provider's terms.
- **Scraping article pages:** only when the site's terms and robots.txt allow it, politely and at low volume.
- **Public datasets:** for evaluation and training, not for a live feed.

- **User-submitted text:** essential. It lets you demo without any scraping risk, and lets examiners paste their own article.

Recommendation: all of them in a layered way: RSS (permitted outlets) + user submissions + a frozen, curated evaluation set.

1.5 Decision 5: show both original and translated views?

Yes. This is your trust feature. Offer a sentence-aligned side-by-side view: clicking a sentence highlights its counterpart. Reviewers and users can spot errors in seconds, and the alignment data feeds your evaluation.

1.6 Decision 6: which extra features are worth it?

Keep	Why
Named entities as chips (people, places, organisations)	Shows context preservation; also powers a verification check
Categories (politics, sports, business, and so on)	Makes the feed navigable; cheap with section labels from feeds
Search and reading history	Required by your brief; PostgreSQL full-text search is enough
Quality flags (low-confidence translation, number mismatch)	Honest uncertainty is a strength
Report a translation error button	Gives human feedback data for your report
Dark mode, copy and share	Cheap polish

Stretch (only if time remains)	Why
Duplicate-story clustering	Real aggregator feel; uses embeddings
Glossary editor for Karnataka names	Improves consistency

Skip	Why
Text-to-speech, mobile app, social login, other languages, push notifications	Scope creep that adds no research or engineering depth

2. The recommended approach: verified translate-then-summarize

This is one pipeline, not several projects:

1. **Ingest** the article (RSS item, fetched page, or pasted text) and store the raw Kannada.
2. **Clean and normalize** Kannada Unicode, split into sentences, keep paragraph structure.
3. **Translate** with IndicTrans2 sentence by sentence, keeping sentence alignment.
4. **Guard the translation:** compare numbers, dates and names between source and output; compute cheap quality signals; raise flags rather than silently trusting.
5. **Summarize** the English translation with an LLM into headline, short summary, key points and key facts, each with evidence sentence IDs.
6. **Verify the summary** against the English translation (entailment checks, entity and number matching) and key facts against the Kannada source.
7. **Enrich:** named entities and category.
8. **Present:** English article, summary panel, original side by side, with flags and sources linked.

Why this is technically meaningful but achievable: the hard parts (Kannada text handling, chunked sentence-level MT, guard checks, grounded generation, verification, evaluation) are all real engineering and all doable with open tools in one semester.

3. Technology research

3.1 Kannada-to-English translation models

Model	Key facts (as of 10 Nov 2026)	Context	Licence and cost	Verdict
IndicTrans2 (1B and 200M distilled)	22 languages; released with the BPCC training corpus and IN22 benchmark; TMLR paper, arXiv 2305.16307	256 tokens per sentence	Checkpoints MIT; data CC0 or CC-BY-4.0 per the AI4Bharat release; free	**Default**
NLLB-200	200 languages; Kannada is kan_Knda	Sentence level	Typically CC-BY-NC ⚠	Baseline
Sarvam-Translate	Gemma3-4B-IT fine-tune; 22 languages; built for structured long documents	8k tokens	Open weights (⚠ check licence); API about ₹20 per 10,000 characters per Sarvam's pricing page; the API caps one request at about 2,000 characters	Strong comparison system

Model	Key facts (as of 10 Nov 2026)	Context	Licence and cost	Verdict
Krutrim-Translate	Distilled IndicTrans2-based model (6 encoder, 3 decoder layers), 10 languages including Kannada; model card reports Kannada-to-English chrF++ of 58.4 on IN22-Gen and 47.3 on IN22-Conv; self-reported, not independent	4,096 tokens	Open on Hugging Face ⚠ check licence	Optional extra baseline
Indic Translate (bodhan-ai)	Gated model, 22 languages, document-level up to 32,768 tokens; claims better dBLEU than Sarvam-Translate (self-reported)	32k tokens	Gated; accept terms ⚠	Watchlist: new and not independently verified
LLM API (Gemini family)	Model names change quickly; the pricing page listed several Flash versions on this date ⚠	Large	A free tier exists for some models, but free-tier content may be used to improve Google's products; check the pricing page	Comparison system and summarizer

Two cautions from recent literature. First, a June 2026 paper (arXiv 2606.29024) adapted IndicTrans2 to conversational text and found chrF gains that a blind human and LLM check did not confirm as real quality gains. Do not trust overlap metrics alone. Second, a

2025 IEEE CONIT paper fine-tuned IndicBART on Kannada-English code-mixed text, a reminder that Kannada news often contains English words written in Kannada script.

Implementation requirements for IndicTrans2: Python 3.10+, PyTorch, Hugging Face Transformers, the IndicTransToolkit (preprocessing and postprocessing), a sentence splitter, and a GPU for the 1B model if you want speed. The 200M distilled model is workable on CPU for background batch jobs.

3.2 Summarization, headline generation and key-information extraction

- **Summarizer, headline and key points:** one LLM call with a JSON schema (headline, 2-3 sentence summary, 3-5 key points, key facts). Use low temperature. Options: a hosted Gemini model, or an open model (Qwen, Gemma, Llama class) served through Ollama or vLLM. Hosted is easier, but see the privacy note below.
- **Key information:** ask for who, what, where, when, how many, each with the supporting sentence IDs, then verify with code.
- **Privacy note:** the Gemini pricing page lists free-tier use as improving Google's products, while paid tier does not. Do not send user-submitted private text through a free tier. Use an open local model for submissions, or tell users plainly.
- **Cost:** a 600-word article is small. At student scale, a few hundred articles cost little or nothing, but model names and prices change, so check the pricing page when you start.

3.3 Named entity recognition

- **Kannada side:** IndicNER (ai4bharat/IndicNER), trained on Naamapadam, which covers 11 Indian languages including Kannada. The paper reports F1 above 80 for most test languages, but the training labels are projected automatically from English (silver labels), so expect noise.
- **English side:** any strong English NER (spaCy transformer model or a Hugging Face BERT NER model).
- **Use:** compare entities across both sides as a translation guard, and show chips in the UI.

3.4 News categorization

Start with weak labels you already have: RSS items and URLs often carry section names (for example sports, national, district). Map those to 8-10 categories. Then train a simple TF-IDF + logistic regression baseline and compare with zero-shot LLM classification on a

hand-labelled sample of about 200 articles. IndicGLUE has a Kannada news-category task ⚠ verify availability and licence before use.

3.5 Evaluation tooling

- **Translation:** sacreBLEU (BLEU and chrF++), COMET (reference-based) and CometKiwi (reference-free quality estimation ⚠ some Unbabel models require accepting a licence on Hugging Face).
- **Summaries:** ROUGE and BERTScore (weak for abstractive text), plus factual-consistency tools such as MiniCheck (arXiv 2404.10774, a 770M model reported to match GPT-4-level accuracy on its benchmark) and AlignScore (355M).
- **Caution:** a 2024 stress-test paper (arXiv 2411.16638) found some NLI-based and specialised factuality metrics overly sensitive to harmless edits, and that LLM-based judges may reflect what the judge model knows rather than what the source says. Use these as flags and always sample human checks.

3.6 Realistic hardware and cost

Need	Zero-cost route	Better route
Run IndicTrans2	200M distilled on CPU for background jobs; 1B on free Colab or Kaggle GPU for batch evaluation	A small rented GPU for the final demo
Summaries	Free-tier API for public articles; local open model for private text	Paid API at a few rupees per day
Database and API	Local Docker	Free-tier hosting
Coding assistant	Antigravity free tier (rate-limited)	Paid plan only if you hit limits

Design rule that saves your demo: never translate while the user waits. Translate and summarize in the background and show ready results; only user-submitted text shows a progress indicator.

4. Application design

Visual direction: clean news-reader look. Tailwind CSS + shadcn/ui components, Inter for English and Noto Sans Kannada for Kannada, one indigo or teal accent, generous whitespace, card layout, dark mode. Kannada glyphs are tall, so give Kannada text a line-height around 1.7 to 1.8 or it looks cramped. Everything responsive down to phone width.

4.1 Screens

Screen	What it shows	Key interactions
Home dashboard	Search bar and a prominent Submit article button; stat strip (articles today, translated, flagged for review, average processing time); Top stories cards (English headline, two-line summary, category pill, source, time, small Kannada headline); category chips; a pipeline status strip showing items in the queue; recent reading history	Click a card to open the article; click a chip to filter
Kannada news feed	Original Kannada headlines with source, time and a status badge (Queued, Translating, Ready, Needs review)	Filter by source, category and date; open any item
English translated feed	Translated headlines with one-line summaries	Toggle card or compact view; same filters

Screen	What it shows	Key interactions
Article detail	Header with English headline (AI-generated, labelled as such), original Kannada headline, source link (Read original), date, category, entity chips. Left column: AI summary, key points, key facts with evidence markers. Main column: reader with a three-way toggle: English, Kannada, Side by side	In side-by-side mode, hovering a sentence highlights its counterpart; flags panel; Report a problem button
Original vs translated comparison	Sentence-aligned two-column view; paragraph breaks preserved; flagged sentences underlined	Click a flagged sentence to see why (number mismatch, low alignment score)
Verification panel (inside detail)	Checks such as: numbers matched 7 of 7; names matched 11 of 12; summary claims supported 5 of 5; method used (LLM or extractive fallback)	Expand to see evidence sentences
Submit article	Paste Kannada text or a URL (permitted sources only), optional title; script check warns if the text is not mostly Kannada; live progress steps	Redirects to the detail page when ready
Search and history	Full-text search over English and Kannada; recent searches; reading history; bookmarks	Filters; clear history

Screen	What it shows	Key interactions
Insights (optional but good for demos and your report)	Charts: articles per category, translation-quality flag rate by source, processing latency, summary verification pass rate	Date range filter

Design every screen with loading skeletons, empty states and friendly errors. Label all AI output clearly: *Machine translation* and *AI-generated summary*.

5. Technical architecture

5.1 Recommended stack

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript + Tailwind + shadcn/ui	Standard, well supported by coding agents, good routing and server rendering
Backend API	Python 3.11 + FastAPI + Pydantic + SQLAlchemy 2 + Alembic	Same language as the ML code; automatic OpenAPI docs help Antigravity and your report
AI worker	Python process in the same repo, loads models once	Avoids model reloads; isolates heavy work from the API
ML	PyTorch, Hugging Face Transformers, IndicTransToolkit, sentence-transformers (LaBSE), spaCy or HF NER	Matches the model ecosystem
LLM access	A provider interface with Gemini and a local open model (Ollama or vLLM) implementations	Swap without touching the pipeline

Layer	Choice	Why
Database	PostgreSQL	Relational data plus JSONB plus built-in full-text search. MongoDB would also work, but your data is relational (articles, sentences, translations, summaries) and you need joins and constraints
Queue	A jobs table in PostgreSQL, polled with SELECT ... FOR UPDATE SKIP LOCKED	No Redis or Celery to set up; easy to explain and debug
Packaging	Docker Compose (db, api, worker, web)	One command to run everything

5.2 System architecture

```
 Browser (Next.js UI)
      |  HTTPS, JSON
      v
 FastAPI  (REST /api/v1) ------------------+
      |  SQL                               |  reads status
      v                                    |
 PostgreSQL  <---- writes results ---- AI Worker
  (articles, sentences,                    |  polls pipeline_jobs
   translations, summaries,                |
   entities, jobs, history)                +--> IndicTrans2 (in process)
      ^                                    +--> LaBSE / QE / NLI checkers (in
process)
      |                                    +--> LLM provider (HTTP: Gemini or
local model)
 Collector (scheduler) -- RSS / fetch ---+--> NER, categorizer
```

How the parts talk. The browser only talks to FastAPI. FastAPI never runs a model; it validates input, writes rows and creates jobs, then returns immediately. The worker claims jobs from the database, runs the AI stages, and writes results back. The frontend polls a status endpoint (or later uses server-sent events) and renders results as they appear. The collector is a scheduled task that inserts new articles and jobs.

5.3 Article lifecycle (data flow)

```
 new -> extracted -> translating -> translated -> summarizing -> verified ->
ready
                         |              |              |             |
                         +--------------+--------------+-------------+-->
failed (retry up to 3)
                                                                    +-->
needs_review (flags too severe)
```

Each arrow is one worker stage. Stages are idempotent: re-running a stage replaces its output rather than duplicating it.

5.4 Database design

```
CREATE TABLE sources (
  id SERIAL PRIMARY KEY, name TEXT NOT NULL, base_url TEXT NOT NULL,
  feed_url TEXT, robots_checked BOOLEAN DEFAULT FALSE, terms_note TEXT,
  active BOOLEAN DEFAULT TRUE);

CREATE TABLE articles (
  id BIGSERIAL PRIMARY KEY,
  source_id INT REFERENCES sources(id),
  origin TEXT NOT NULL CHECK (origin IN ('rss','user','dataset')),
  url TEXT UNIQUE, title_kn TEXT, body_kn TEXT NOT NULL,
  content_hash CHAR(64) UNIQUE,
  section_label TEXT, published_at TIMESTAMPTZ,
  fetched_at TIMESTAMPTZ DEFAULT now(),
  status TEXT NOT NULL DEFAULT 'new',
  session_id TEXT);                       -- anonymous owner for user
submissions

CREATE TABLE sentences (                  -- source side, split once
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  idx INT NOT NULL, paragraph_idx INT NOT NULL, text_kn TEXT NOT NULL,
  UNIQUE (article_id, idx));

CREATE TABLE translations (               -- one row per backend/version
tried
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  backend TEXT NOT NULL, model_version TEXT NOT NULL,
```

```
  title_en TEXT, guard_report JSONB,
  is_active BOOLEAN DEFAULT TRUE, created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE sentence_translations (
  translation_id BIGINT REFERENCES translations(id) ON DELETE CASCADE,
  sentence_id BIGINT REFERENCES sentences(id) ON DELETE CASCADE,
  text_en TEXT NOT NULL, labse_sim REAL, qe_score REAL,
  flags JSONB DEFAULT '[]',
  PRIMARY KEY (translation_id, sentence_id));

CREATE TABLE summaries (
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  translation_id BIGINT REFERENCES translations(id),
  model TEXT NOT NULL, prompt_version TEXT NOT NULL,
  method TEXT NOT NULL CHECK (method IN ('llm','extractive_fallback')),
  headline_en TEXT, summary_en TEXT,
  key_points JSONB,   -- [{text, evidence:[sentence idx]}]
  key_facts JSONB,    -- {who:[], what:[], where:[], when:[], numbers:[]}
  verification JSONB, -- per-claim support scores, entity and number checks
  created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE entities (
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  lang CHAR(2) NOT NULL, text TEXT NOT NULL, norm_text TEXT,
  type TEXT NOT NULL, sentence_idx INT);

CREATE TABLE categories (id SERIAL PRIMARY KEY, slug TEXT UNIQUE,
  name_en TEXT, name_kn TEXT);
CREATE TABLE article_categories (
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  category_id INT REFERENCES categories(id),
  confidence REAL, method TEXT, PRIMARY KEY (article_id, category_id));

CREATE TABLE pipeline_jobs (
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id) ON DELETE CASCADE,
  stage TEXT NOT NULL, state TEXT NOT NULL DEFAULT 'queued',
  attempts INT DEFAULT 0, last_error TEXT,
  locked_at TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE search_history (id BIGSERIAL PRIMARY KEY, session_id TEXT,
```

```
  query TEXT, created_at TIMESTAMPTZ DEFAULT now());
CREATE TABLE reading_history (id BIGSERIAL PRIMARY KEY, session_id TEXT,
  article_id BIGINT REFERENCES articles(id), opened_at TIMESTAMPTZ DEFAULT
now());
CREATE TABLE bookmarks (session_id TEXT,
  article_id BIGINT REFERENCES articles(id), PRIMARY KEY (session_id,
article_id));
CREATE TABLE feedback (id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles(id), sentence_id BIGINT,
  kind TEXT, comment TEXT, created_at TIMESTAMPTZ DEFAULT now());
```

Use the `simple` text-search configuration for Kannada (PostgreSQL has no Kannada stemmer) and `english` for translated text. Skip user accounts: an anonymous `session_id` cookie is enough for history and bookmarks and keeps the project focused. Evaluation tables are added later (prompt P13).

5.5 API surface (REST, prefix /api/v1)

Method and path	Purpose
GET /articles	List with filters: source, category, status, date, page
GET /articles/{id}	Full detail: Kannada text, sentences, active translation, summary, entities, flags
POST /articles/submit	Submit Kannada text (or permitted URL); returns article id and job id
GET /articles/{id}/status	Stage and progress for polling
POST /articles/{id}/retry	Re-run a failed stage
GET /search?q=	Full-text search; records search history
GET /history/search, GET /history/reading, POST /history/clear	History
POST /bookmarks, DELETE /bookmarks/{id}, GET /bookmarks	Bookmarks
GET /categories, GET /sources	Reference data
POST /feedback	Report a translation or summary problem

Method and path	Purpose
GET /stats	Dashboard and insights numbers
GET /health	Liveness and model status

5.6 Folder structure

```
kannada-news-ai/
  docker-compose.yml  .env.example  Makefile  README.md
  docs/            architecture.md  ethics.md  eval-protocol.md
  apps/
    web/                        (Next.js)
      src/app/                  page routes: /, /kannada, /english,
/article/[id], /submit, /search, /history, /insights
      src/components/           ArticleCard, SideBySide, SummaryPanel,
FlagBadge, EntityChips, ...
      src/lib/api.ts            typed API client
    api/                        (FastAPI)
      app/main.py  app/config.py  app/db.py
      app/models/               SQLAlchemy models
      app/schemas/              Pydantic schemas
      app/routers/              articles.py search.py history.py feedback.py
stats.py
      app/services/             article_service.py search_service.py
      alembic/  tests/
  services/
    worker/
      worker.py                 job loop
      stages/                   extract.py preprocess.py translate.py
guard.py summarize.py verify.py enrich.py
    ml/
      kannada_text.py           normalization, sentence split, digits
      translation/              base.py indictrans2.py sarvam.py llm.py
mock.py
      llm/                      base.py gemini.py local.py
prompts/summarize_v1.txt
      verification/             numbers.py entities.py nli.py extractive.py
      nlp/                      ner.py categorize.py
    collector/
      collector.py  sources.yaml  robots.py  extract.py
  eval/
```

```
    datasets/  scripts/  results/  annotation/
  scripts/                      seed.py  bench_translation.py
demo_articles.json
  .github/workflows/ci.yml
```

6. The AI pipeline in detail

6.1 What happens to one article, stage by stage

Stage 1: Text extraction. For RSS items, take the feed summary, then fetch the page only if permitted and extract the main text with a library such as trafilatura, removing menus, ads, related-story boxes and bylines. Check that the result is mostly Kannada by measuring the share of characters in the Kannada Unicode block (U+0C80 to U+0CFF). Reject or flag pages that are mostly English or too short. Deduplicate by content hash, and later by embedding similarity for near-duplicates.

Stage 2: Preprocessing. Normalize to Unicode NFC. Be careful with the zero-width joiner and non-joiner characters: they are meaningful in Kannada conjuncts, so do not strip them blindly. Convert Kannada digits (೦ to ೯) to ASCII digits only in a *copy* used for number checking, not in the text shown to users. Split into paragraphs, then sentences (the Indic NLP Library has Indic-aware sentence splitting). Keep paragraph and sentence indices, because they power the side-by-side view and the evidence citations. Split any sentence longer than the model limit at clause boundaries.

Stage 3: Kannada-to-English translation. Run IndicTrans2 (Indic-to-English) with its toolkit's preprocessor and postprocessor, in batches, beam size around 5. Translate the headline separately, because headlines are telegraphic and need different handling. Store each sentence's translation with its alignment to the source sentence. Because IndicTrans2 sees one sentence at a time, it can drop pronoun reference across sentences, so the guard stage matters.

Stage 4: Context and meaning preservation. You cannot make a sentence-level model understand context, so you *measure* what was preserved:

Guard	How it works	If it fails
Number guard	Extract all numbers (including Kannada digits, lakh and crore amounts, percentages) from source and translation; compare as sets after normalization	Flag sentence: numbers differ
Date guard	Same idea for dates and month names	Flag
Name guard	Run IndicNER on the Kannada and English NER on the translation; match entities by transliteration similarity and a Karnataka glossary (Bengaluru, Mysuru, party names, ministries, and so on)	Flag: entity missing or changed
Alignment guard	LaBSE similarity between each Kannada sentence and its English translation; low similarity suggests omission, addition or hallucinated translation	Flag below a threshold you tune on your sample
Quality estimation	A reference-free metric such as CometKiwi per sentence	Use only as a relative flag (see caution below)
Length-ratio guard	Extreme source-to-output length ratios suggest truncation or repetition	Flag

Caution on quality estimation: a 2026 benchmark (IndicQE-APE, arXiv 2608.16344) found that reference-free scores are not comparable across language pairs, and it does not include Kannada among its pairs. Treat the score as an uncalibrated warning light and validate it against human ratings on your own Kannada sample (this becomes a research question in Section 8).

Stage 5: Summarization, headline, key points. Send the *English translation as numbered sentences* to the LLM with a strict prompt and JSON schema. Template for `prompts/summarize_v1.txt`:

```
You are a news editor. You will receive a news article as numbered sentences
[S1], [S2], ...
Use ONLY information stated in these sentences. Do not add background,
causes, opinions,
names, numbers or dates that are not in the text. Keep the original
journalistic tone.
If the article does not say something, do not say it.

Return JSON only, matching this schema:
{
  headline: string (max 14 words, factual, no clickbait),
  summary: string (2-3 sentences),
  key_points: [ { text: string, evidence: [sentence numbers] } ]  (3 to 5
items),
  key_facts: { who: [..], what: [..], where: [..], when: [..], numbers: [..]
},
  uncertain: [ strings for anything unclear or contradictory in the text ]
}
Every key point MUST list the sentence numbers that support it.
```

Set temperature low, validate the JSON with Pydantic, and retry once on invalid output.

Stage 6: Verification (Section 6.3 explains each layer).

Stage 7: Enrichment. NER on both sides, category assignment (feed section label first, classifier second), search indexing.

Stage 8: Final presentation. The UI shows the English article, summary panel, evidence markers, flags and the original, with *machine-generated* labels and a link to the source.

6.2 Is this the best pipeline? Evaluation and improvements

Your proposed workflow (extract, preprocess, translate, preserve meaning, summarize, headline, key points, present) is a sound backbone. Improvements I recommend:

1. **Make “context and meaning preservation” a measurable stage** with the guards above, not a vague goal.
2. **Add a verification stage after summarization.** This is what turns a demo into an engineering project.
3. **Keep sentence IDs through the whole pipeline.** That enables evidence citations, side-by-side highlighting and fine-grained evaluation.

4. **Verify against the Kannada source too.** A summary can be perfectly faithful to a bad translation. Checking key facts against the original catches that.
5. **Run NER, categorization and headline generation in parallel** where possible; they do not depend on each other.
6. **Fallback ladder:** if verification fails, regenerate once with feedback, then fall back to an extractive summary labelled as such, and mark the article for review.
7. **Do not add an LLM "polish" pass by default.** In the IndicQE-APE benchmark, leaving the machine translation unedited beat every prompted post-editing system on three of four language pairs under surface metrics (though a reference-free metric disagreed). Test whether polishing helps in your ablation before adding it.

6.3 Avoiding hallucinations: seven layers

1. **Closed-book prompt.** Only the supplied sentences may be used; unknowns go in an `uncertain` field.
2. **Evidence requirement.** Each key point must cite sentence numbers; reject any citing numbers that do not exist.
3. **Deterministic entity and number check.** Every name, number and date in the summary must appear (after normalization and fuzzy matching) in the English translation. This catches the most damaging hallucinations cheaply and is fully explainable.
4. **Claim-level entailment check.** Split the summary into claims and score each against the cited sentences with MiniCheck or AlignScore. Reject below your tuned threshold. Remember these tools can be fooled and are not perfect judges.
5. **Source-side check.** Verify key_facts (names, numbers, dates, places) against the *Kannada* text using the Kannada NER and number extraction, which guards against translation-induced errors.
6. **Fallback.** Regenerate with the failed claims listed; if that fails, use the extractive summary.
7. **Human feedback loop.** The Report a problem button stores feedback for analysis, and a sampled human check measures the real hallucination rate for your report.

6.4 Honest limits

Sentence-level translation can mishandle idioms, headline style and cross-sentence references. Names can be transliterated inconsistently. Verification tools are imperfect, and none of this makes the system suitable for legal, medical or safety-critical use. Say so

in your report and in the UI.

7. Datasets and news collection

7.1 Collecting live Kannada news

- **RSS:** look for a feed link in each site's page header (a link of type rss or atom) or footer. Use the `feedparser` library. I could not verify which Kannada outlets publish usable feeds or what their terms allow ⚠: check each site yourself and record the result in `sources.yaml` (the `robots_checked` and `terms_note` columns exist for this).
- **Politeness rules:** identify your crawler in the User-Agent string, obey robots.txt, limit to one request every few seconds per site, cache with ETag and Last-Modified, and fetch full pages only when permitted. Never touch paywalled content.
- **Outlets to check first:** Prajavani, Udayavani, Kannada Prabha, Vijay Karnataka, Vijayavani, Samyukta Karnataka. Also consider government or public-information sources, which are often more reusable (the PMIndiaSum authors note the Prime Minister of India website allows content redistribution).
- **News APIs:** aggregator APIs sometimes offer a language filter for Kannada, but typically return headlines and snippets with licence and quota limits ⚠ verify each one. Treat them as discovery tools.
- **User-submitted text:** a paste box plus an optional URL. This is your safest demo path.

7.2 Public datasets for training and evaluation

Dataset	What it is	Use in your project	Notes
BPCC (with IndicTrans2)	About 230 million bitext pairs for 22 Indian languages (mined plus human)	Reference corpus; fine-tuning if you ever need it	Mixed licences: mined and back-translation parts CC0, newly added human seed parts CC-BY-4.0

Dataset	What it is	Use in your project	Notes
IN22-Gen and IN22-Conv	Multi-domain n-way benchmarks (1,024 and 1,503 sentences) covering 22 Indian languages including Kannada	Standard Kannada-to-English test set	CC-BY-4.0; the Hugging Face repo is gated (accept terms). Wikipedia and web text, not mainly news
FLORES-200	3,001 professionally translated sentences per language across 200 languages	Second standard test set	⚠ check licence and that your models did not train on it
Samanantar	Large parallel corpus for 11 Indic languages	Background training data of many models	Mined, so noisier
PMIndiaSum	76,680 monolingual and 620,336 cross-lingual document-headline pairs across 14 Indian languages including Kannada (EMNLP 2023)	Evaluate headline generation and cross-lingual summaries; government news domain	Source site permits redistribution per the paper
CorIL (arXiv 2509.19941, 2025)	772,000 sentence pairs across 11 Indian languages including Kannada, three domains	Extra Kannada-English test or training data	Check licence on its dataset page ⚠
Naamapadam	NER data for 11 Indic languages including Kannada; CC0	Train or evaluate NER	Training labels are projected (silver); small human test sets
IndicGLUE news categorization	Kannada news-category task	Train or evaluate the classifier	⚠ verify availability and licence

Gap you should fill yourself: none of these is a Kannada *news* article benchmark with full-article translations and reference summaries. Building a small, carefully documented one (Section 8) is the most valuable thing you can create. It also avoids train-test contamination because you will use articles published after the models were released.

7.3 Storage and preprocessing

Store raw HTML or text (for provenance), cleaned Kannada text, sentence splits, translations per model version, summaries per prompt version, and a content hash. Keep the raw material private; the public demo should show short excerpts, your machine-generated output, and a link to the original. Log model names, versions and prompt versions on every output so experiments are reproducible.

7.4 Copyright and ethics checklist

- Respect each site's terms and robots.txt; prefer feeds, official sources and user-submitted text.
- Do not republish full copyrighted articles publicly. Show headline, short excerpt and your derived summary with attribution and a Read original link. Keep full text for private research processing only, and check your university's guidance.
- Label everything *machine translation* or *AI-generated*. Machine output can mislead on sensitive stories (politics, communal incidents, crime), so display a short accuracy notice.
- Do not "correct" framing: the system should preserve the outlet's tone. Check in your evaluation that summaries do not add or remove loaded language without saying so.
- Privacy: user-submitted text may be personal. Do not send it to a service that trains on inputs (the free Gemini tier lists use of content to improve products). Delete submissions on request and say how long you keep them.
- Credit datasets and models with their licences in a LICENSES file, and respect non-commercial restrictions.
- Your human evaluators should be informed volunteers or paid, with clear instructions.

8. Research potential

8.1 Research problem

How well does a translate-then-summarize pipeline serve readers who need English versions of Kannada news, and can an explicit verification layer make the English summaries more faithful to what the Kannada article actually says?

8.2 Research gap (what I found, and how sure I am)

In my searches I did not find a 2024 to 2026 paper that evaluates Kannada-to-English *news* translation and summarization end to end with source-grounded faithfulness checking. My searches were not exhaustive, so confirm with your own literature search before claiming a gap. The pieces that point to the gap:

1. **Domain.** Standard Indic benchmarks are mostly Wikipedia, web and conversational text. One model card (Krutrim-Translate) reports a notable Kannada-to-English drop from IN22-Gen to IN22-Conv, which shows domain matters for Kannada. News has its own style (headlines, datelines, names, lakh and crore amounts).
2. **Pipeline evidence is general, not Kannada-specific.** Varab & Hardmeier (2024) support translate-then-summarize across 39 languages, and a 2026 shared-task system paper (arXiv 2601.09059) used a translate-first approach for Indic-language dialogues including Kannada. Neither studies Kannada news or measures faithfulness to the original text.
3. **Error propagation is rarely measured.** Faithfulness metrics compare a summary to the *translation*. If the translation is wrong, the summary can score well and still be wrong.
4. **Reference-free quality estimation for Kannada is unvalidated.** IndicQE-APE (2026) consolidates quality-estimation data for six Indian languages plus Estonian, Nepali and Sinhala, and does not include Kannada. It also shows scores are not comparable across languages. Validating a QE metric for Kannada against human judgement is a small but genuine contribution.
5. **Post-editing is not automatically helpful.** The same paper found unedited MT beating prompted post-editing under surface metrics, so “add an LLM polish step” is a testable claim, not a given.
6. **Factuality metrics themselves are shaky.** A 2024 study found some are overly sensitive to benign edits and LLM judges can reflect their own knowledge. Combining automatic checks with a human audit is therefore necessary and worth reporting.

8.3 Research questions

- **RQ1.** Which translation systems (IndicTrans2 1B and 200M, NLLB-200, Sarvam-Translate, an LLM API) work best on Kannada news, by automatic and human measures?
- **RQ2.** Does translate-then-summarize (P1) beat summarize-then-translate (P2) and one-shot LLM (P3) on faithfulness and usefulness for Kannada news?

- **RQ3.** Does the verification layer reduce unsupported content in summaries, and at what cost to coverage?
- **RQ4.** Do sentence-level quality scores (CometKiwi, LaBSE similarity) predict human-judged translation errors for Kannada?
- **RQ5 (optional).** Does an LLM post-editing pass improve or harm fidelity?

8.4 Methodology

Dataset you build (KannadaNews-Eval). Collect about 150 to 200 Kannada articles from at least four outlets across eight categories, published *after* the models you test were released (this limits train-test contamination). Record source, date, category and licence note. For 50 to 100 of them, obtain an independent English reference translation from bilingual volunteers or paid translators. Be careful: if the references are made by post-editing the machine output, they are anchored on that output and reward conservative systems (a point made in IndicQE-APE). For about 50 articles, also write reference summaries (headline plus two to three sentences). Release only what the licences allow, such as article URLs and your annotations.

Systems.

Role	Systems
Translation baselines	IndicTrans2-1B, IndicTrans2-200M distilled, NLLB-200 (a distilled size), Sarvam-Translate, a Gemini model prompted to translate
Pipelines	P1 translate then summarize; P2 summarize in Kannada then translate; P3 one-shot LLM
Verification ablation	None; entity and number guard only; guard plus entailment check; full (with source-side check)
Summarizer	One fixed LLM across conditions, so differences come from the pipeline

Automatic metrics.

Metric	Measures	Tool	Caveat
BLEU	Word n-gram overlap with a reference	sacreBLEU (report the signature)	Poor for morphologically rich languages; synonyms are penalised
chrF++	Character and word n-gram F-score	sacreBLEU	Better than BLEU for Kannada, still surface-based
COMET	Learned semantic similarity to a reference	Unbabel COMET models	Trained on limited Kannada judgements; check language coverage
CometKiwi	Reference-free quality estimate	Unbabel CometKiwi	Not comparable across languages; validate for Kannada (RQ4)
ROUGE-1, ROUGE-2, ROUGE-L	Overlap with reference summaries	rouge-score	Weak for abstractive summaries; report but do not rely on it
BERTScore	Embedding similarity	bert-score	Does not detect factual errors
Entailment-based faithfulness	Is each summary claim supported by the text	MiniCheck, AlignScore	Imperfect; sample human checks
Entity and number precision	Share of summary entities and numbers found in the source	Your code	Deterministic and explainable

Human evaluation.

- *Annotators:* two or three bilingual (Kannada and English) volunteers or paid annotators, trained with a one-page guideline and a pilot of about 10 items.
- *Translation:* on a sample of about 40 articles (10 sentences each, including flagged and unflagged ones), rate adequacy and fluency (1 to 5) and tag errors using a light MQM-style scheme: mistranslation, omission, addition, wrong name, wrong number, untranslated word, grammar.

- *Summaries:* judge each sentence as supported, partly supported or unsupported by (a) the Kannada source and (b) the English translation; also rate coverage of the main event and readability. The difference between (a) and (b) measures error propagation.
- *Agreement:* report Cohen's kappa or Krippendorff's alpha, and resolve disagreements by discussion.
- *Blinding:* hide which system produced each output and randomise order.

Factual-consistency definition. Hallucination rate = unsupported summary sentences divided by all summary sentences, reported against both the Kannada source and the English translation, with severity (minor wording versus wrong name, number or event).

Statistics. Use paired bootstrap resampling for metric differences, report confidence intervals, and keep sample sizes honest. With small human samples, claim trends rather than significance.

8.5 Related papers worth reading (2024 to 2026, plus essential foundations)

Year	Paper	Why it matters
2024	Varab and Hardmeier, *With Good MT There is No Need For End-to-End* (arXiv 2409.00414)	Evidence for translate-then-summarize into English
2024	Tang, Laban and Durrett, *MiniCheck* (arXiv 2404.10774)	Small, efficient fact-checker for grounding checks
2024	*Do Automatic Factuality Metrics Measure Factuality?* (arXiv 2411.16638)	Why you must audit metrics with humans
2024	*Towards LLM-driven Reference-less Translation Evaluation for English and Indian Languages* (arXiv 2404.02512)	Reference-free evaluation for Indian-language MT; read before citing
2025	CorIL parallel corpus (arXiv 2509.19941)	Kannada-inclusive parallel data; compares NLLB-200, IndicTrans2 and other models after fine-tuning
2025	IndicBART for Code-Mixed Kannada-English Translation (IEEE CONIT, June 2025)	Kannada-specific, code-mixed

Year	Paper	Why it matters
2025	AlignCheck (arXiv 2512.03634)	Interpretable factual-consistency framework
2026	Shared-task system description, NLPAI4Health 2025 (arXiv 2601.09059)	Translate-first design across Indic languages including Kannada
2026	Conversational Domain Adaptation of IndicTrans2 (arXiv 2606.29024)	Warns that chrF gains may not mean real quality gains
2026	IndicQE-APE (arXiv 2608.16344)	Indic quality estimation limits; do-nothing post-editing baseline
Foundation	IndicTrans2 (arXiv 2305.16307); PMIndiaSum (arXiv 2305.08828); IndicMT Eval (arXiv 2212.10180); AlignScore (arXiv 2305.16739); Naamapadam (arXiv 2212.10168)	Models, datasets and metric meta-evaluation you will cite

8.6 Realistic novelty and contribution

Be straightforward with your examiners: you are **not** proposing a new translation model or a new summarization algorithm. The novelty is in the **combination and evaluation**. In descending order of research value:

1. A small, documented **Kannada news evaluation set** with human judgements.
2. A **pipeline comparison and verification ablation** on that set, including source-grounded faithfulness (error propagation).
3. **Validation of QE signals** for Kannada (RQ4).
4. The **open, working system** that makes all of it reproducible.

This is a solid scope for a final-year project report and could support a workshop or regional conference paper if your evaluation is careful. It is unlikely to be a top-tier venue paper unless you extend to larger-scale human evaluation or a new method. Overclaiming is the fastest way to lose credibility, so state limits openly.

9. Development through Google Antigravity

9.1 How to work with Antigravity

Antigravity is Google's agent-first coding environment: agents can plan, edit code, run terminal commands and use a browser to test, and they show their work as Artifacts such as plans and screenshots. It launched in November 2025 and was expanded in 2026. Plans, limits and prices have changed more than once and sources disagree ⚠, so check what your account includes. Free usage is rate-limited, so work in small scoped tasks.

Working rules that save quota and prevent mess:

1. **Paste prompts one at a time, in order.** Do not start the next until the tests of the current one pass.
2. **Ask for a plan first.** Every prompt below says: *produce a short plan and wait for my approval before editing*. Read the plan; it takes a minute and catches wrong assumptions.
3. **Review the diff and run the tests yourself.** Agents can claim success while a test is failing.
4. **Commit after every green prompt** (`git commit -m "P5 translation service"`) so you can roll back.
5. **Never paste real API keys into prompts.** Use `.env` files and `.env.example`.
6. **Keep a small set of fixed demo articles** (three Kannada articles in `scripts/demo_articles.json`) to test every stage the same way.
7. **Mock first, real model second.** The translation and LLM interfaces get a mock implementation, so the frontend and tests work without a GPU or API key.

9.2 Milestone roadmap (about 12 weeks)

Week	Milestone	Done when
1	M0 Setup	Repo, Docker Compose, empty API and web apps run; workspace rules saved
2	M1 Database and API skeleton	Migrations apply; seeded demo articles visible through the API
2-3	M2 Frontend foundation	Layout, design system and pages render mock data

Week	Milestone	Done when
3-4	M3 News collection	Collector fetches permitted RSS items and stores clean Kannada text; user submit works
4-5	M4 Translation	IndicTrans2 translates demo articles sentence by sentence with guards and flags
6-7	M5 Summarization and verification	JSON summaries with evidence; verification and fallback working
7	M6 Orchestration	Worker runs the full pipeline automatically with retries
8	M7 Extra NLP	NER, categories, search and history working
7-10	M8 Evaluation	KannadaNews-Eval collected, metrics scripts run, human annotation under way
10	M9 Testing and hardening	Test suite, end-to-end tests and bug fixes done
11	M10 Deployment	Public demo or recorded demo running from containers
12	M11 Report and presentation	Results tables, error analysis, limitations, demo script

9.3 Order of prompts

Prompt	Topic	Maps to your list
P0	Workspace rules (paste once)	1 Project setup
P1	Repository and environment setup	1 Project setup
P2	Database models and migrations	4 Database integration
P3	Backend core API	3 Backend development
P4	Frontend foundation and mock data	2 Frontend development
P5	Frontend pages	2 Frontend development
P6	Kannada news collection	5 News collection
P7	Kannada text processing	5 and 6

Prompt	Topic	Maps to your list
P8	Translation service	6 Translation integration
P9	Translation guards	6 Translation integration
P10	Summarization service	7 Summarization integration
P11	Faithfulness verification	7 Summarization integration
P12	Pipeline worker	3 and 7
P13	NER, categories, search, history	8 Additional NLP features
P14	Evaluation harness	Research support
P15	Testing and debugging	9 Testing and debugging
P16	Deployment	10 Deployment